An Algorithm for HUM 110 Assignment

A Thesis
Presented to
The Division of Mathematical and Natural Sciences
Reed College

In Partial Fulfillment
of the Requirements for the Degree
Bachelor of Arts

Anna Wu

May 2026

Approved for the Division
(Computer Science)

Harper Knittel

Acknowledgements

List of Abbreviations

HUM	Humanities
P	Polynomial
NP	Nondeterministic Polynomial
DA	Deferred Acceptance
1M	1-sided matching
F1M	Group fair 1-sided matching
ISF1M	Intra-group stable, group fair 1-sided matching
FCFS	First Come, First Served
WT	Weighted Trading

Table of Contents

Introduction **1**

Chapter 1: Background **3**
1.1 Analyzing Algorithms 3
1.1.1 Big-O 3
1.1.2 P vs NP 6
1.2 Graph Theory 7
1.3 Mechanism Design 9
1.3.1 Stable Matching 9
1.3.2 Fair Division 10
1.4 Adding Constraints 11
1.4.1 Quotas and Diversity 12
1.4.2 Group Fairness 12
1.5 Related Work 13
1.5.1 Two-sided matching 13
1.5.2 One-sided matching 13

Chapter 2: Formalizing the Problem **15**
2.1 The Model 15
2.2 The Challenge of Stability 19

Chapter 3: Developing an Algorithm **23**
3.1 First Come, First Served 23
3.2 A Greedy Approach 24
3.3 The Weight of the Situation 27

Chapter 4: Experiments **31**
4.1 Data 31
4.2 Implementation 31
4.2.1 Synthesizing Preferences 32
4.2.2 Notes on the Algorithms 33
4.3 Results 33
4.3.1 Is it fair? 34
4.3.2 Is it stable? 35

4.3.3 Is it balanced? 37

Conclusion **39**

References **41**

List of Figures

1.1 Example starting configuration of a solitaire game. 6
1.2 Simple graph with 6 vertices and 7 edges. 8
1.3 Directional graph from Figure 1.2 with weights. 8
1.4 Bipartite directional graph illustrating the Stable Marriage problem. . 10
1.5 Cake cutting example where the left half of the cake is vanilla, the right half is chocolate, v_i nodes represent vanilla people, and c_i represents chocolate people. In this case, c_1 and v_3 would envy each other. . . . 11

2.1 Graph illustrating an F1M, where each group of students, X, Y, and Z, is represented by a different color. 17
2.2 Graph illustrating weighted edges for student x_1. 18

4.1 Biadjacency matrix representing 3 students and 3 seats. 33
4.2 (a) Comparison of minimum number of students of each group over classes in all trials. (b) Comparison of maximum number of students of each group over classes in all trials. 34
4.3 Comparison between number of students by group assigned to each class over varying preference list lengths and years of sample data. . . 35
4.4 Comparison of inter-group blocking pairs present in each trial. 36
4.5 Comparison of number of unmatched students in each trial. 36
4.6 Comparison of average rank of assignment in each trial. 37
4.7 (a) Comparison between sample years of distribution of *grouped* students to class sections over all trials. (b) Comparison between sample years of distribution of all students to class sections over all trials *without group fairness constraints*. 37

Abstract

The preface pretty much says it all.

Introduction

The Reed College Registrar's Office handles all logistics for scheduling at the school: the schedule of classes, final exam scheduling, classroom scheduling, etc. The one schedule Reed students are responsible for making is their own class schedule each year, which proves a difficult task for some. For incoming freshmen especially, navigating the intricacies of the class schedule in the face of their new Reed lives can lead to some scheduling conflicts that stump even the strongest of academic advisors. The plague of class scheduling in general is not unique to Reed College, though there is one friction point that is specific to the school and the first-years in particular: Humanities 110.

Humanities 110, or more notoriously HUM 110, is a graduation requirement for all Reed students. It is a conference-style class designed to be taken by all freshmen each year to introduce them to humanistic inquiry in preparation for the rest of their studies at Reed. The key scheduling aspect of HUM 110 is that it is a year-long course in a semester-based academic calendar. The Registrar has expressed that every year they are faced with freshmen who struggle to fit the same HUM 110 section into both semesters of their first-year schedule. When I came to them as an aspiring CS senior thesis student with an interest in scheduling algorithms, they inquired about a potential solution to this HUM 110 complication: is it possible to let freshmen choose all their classes for the year except for HUM 110, and then sort them into HUM 110 sections based on the schedules they construct? Can we put even 80% of students into a year-long section that doesn't conflict with the rest of their schedules?

This thesis is presented in collaboration with the Reed College Registrar to come up with an algorithm that assigns Reed freshmen to year-long HUM 110 sections based on the rest of their chosen class schedules. Over the course of this thesis, we present an algorithm that stands as a hypothetical solution to this problem. Of course, we are not the first in the field of computer science to study scheduling algorithms, or even class scheduling more specifically. There exists plenty of software that will assign class schedules, likely even for more specific cases such as HUM 110. However, The Registrar informed me that most of their scheduling is done by hand, due to the expense of these software programs as well as the overhaul of transitioning to an entirely new software-based system. The benefit of Reed being a relatively small school is that the scale of their scheduling problems is small enough for the manual strategy to be effective, if more time-consuming. We hope to create a specialized

algorithm that would be easy and free to implement and hopefully reduce the amount of work The Registrar has in balancing freshmen into HUM 110 sections every year.

Because class scheduling is an extensively studied field, we have formulated a very specific problem with a very specific goal to build off of that research in contribution to the Reed community. We want to assign students to just one year-long class amidst the rest of their chosen classes, and we want to do it in a way that both satisfies students and maintains the philosophy behind HUM 110. The course serves as a foundation for every student's course of study at Reed; as part of that, it should foster an environment where students are surrounded by other students with a variety of academic interests, from language and literature to social sciences to hard sciences to arts. So, we have developed an algorithm that assigns students to HUM 110 sections based on their ranked preferences over the sections, and does so in a way that balances students of different disciplines into sections of diverse academic interests.

In this paper, we discuss the formal mathematical guarantees of our algorithm as well as evaluate its performance against our goals and the existing class sign-up system. Chapter 1 provides necessary background information for understanding the mathematical analysis and construction of the algorithm, as well as more information on the existing literature on the subject. Chapter 2 defines the formal model for our problem which allows us to prove important features our algorithm has to accomplish our goals. Chapter 3 discusses how we developed the algorithm and provides pseudocode outlining how it works. Chapter 4 presents the experiments we ran to test the efficacy of our algorithm as well as what results were produced.

Chapter 1

Background

1.1 Analyzing Algorithms

An *algorithm* is a list of specific instructions written to achieve some goal. The word "algorithm" has become an increasingly prevalent phrase on the internet with the popularity of social media, and it is often framed as an opaque force of technology that controls what content users are shown. While those social media algorithms exist, they are only one type of algorithm. A cookie recipe is an algorithm, providing instructions a baker follows to yield a batch of cookies upon completion. GPS devices provide algorithms for driving from one location to another. In computer science, an algorithm is often implemented as a sequence of lines of code that perform a computation, usually mathematical calculations or processing data. Because algorithms are so versatile, and there are so many different ways of accomplishing one goal, we analyze algorithms by their efficiency: how many resources they use and how much time they take to complete the task.

1.1.1 Big-O

To analyze efficiency, we have to define what we mean for an algorithm to be "efficient." For the sake of this thesis, we will focus on the time aspect of efficiency. The amount of time an algorithm takes to execute from start to finish is known as the *running time* of an algorithm. We can mathematically formalize running time with what's known as *Big-O* notation.

Big-O notation is one form of *asymptotic* analysis, or analysis that takes place as the size of the input becomes infinitely large. If we want to know how many resources an algorithm will require (resources can mean power, time, storage space, etc.), we want to know how it performs in the *worst case* scenario, as any other case will use fewer resources. In many scenarios, the worst case is abstractly thought to be an infinitely large input, representing how a realistically large input often requires more computational resources. So, worst case analysis as a facet of asymptotic analysis

allows us to guarantee an algorithm scales well for arbitrarily large inputs. For instance, if I am following an algorithm to bake cookies and the steps involve rolling out the dough, mixing, and measuring ingredients by hand, the algorithm works well for me to make a batch of 24 cookies. However, if I start a popular cookie franchise and suddenly I need 2,400 cookies a day, my original algorithm is not going to be the most efficient way to make that many cookies. While it may make sense when baking cookies to handle these two cases differently, in computer science problems data commonly scales to very large sizes, so it is helpful to know our algorithms will still be efficient in those cases.

We *estimate* running time through worst case asymptotic analysis. In Big-O notation, we write that for some algorithm called f, $f(n) = O(g(n))$. This is read "$f(n)$ is big O of $g(n)$," where f on input n will take **at most** $c \cdot g(n)$ time to run for some constant c. The O signifies the "at most" in that statement, or more formally it denotes an *upper bound* on the asymptotic running time. A "constant" in this case is simply a positive number that does not change as n changes, and in Big-O we denote $f(n) = O(c \cdot g(n)) = O(g(n))$ to indicate that we are only concerned with how f grows as n becomes large. We can think of c in the context of running an algorithm on actual computers: a computer from 1990 (when microprocessors were only just becoming accessible to the public) will have much less processing power than a computer from 2020 with 30 more years of microprocessor innovation. So, c will be larger for the 1990 computer than the 2020 computer. However, both computers will take more time as n grows large regardless of c, which is why we focus on n in Big-O. It is also relevant to note that "time" is purposefully abstract in the term "running time;" we just use it to mean "some unit of time." We can measure units in seconds, milliseconds, operations, etc., so c also represents the fact that we are not always using the same unit of measurement when performing f. However, similarly to the processing power of different computers, we will use more of our unit of measurement for any unit as n grows large. For example, if it takes one minute to make one batch of cookies, it will take 10 minutes to make 10 sequential batches of cookies. Alternatively, if it takes one hour to make a batch of cookies, it will take 10 hours to make 10 sequential batches of cookies. In both cases, if we are committed to making cookies, the amount of time the cookies take will grow with the number of batches (n) we want to perform.

Running time can be classified into different categories. The most relevant categories to this thesis are constant time, polynomial time, and exponential time. In very, very simplified terms, for a basic understanding of these categories, we can consider constant time to be fast, polynomial time to be medium speed, and exponential time to be slow. An algorithm can belong to one these different categories based on its worst case running time.

Constant time, denoted $O(1)$, can be abstractly thought of as any operation that takes the same fixed amount of time regardless of the input size. Constant time is the fastest category of running time. Say our friend Alice chooses to eat Excellent Cuisine, her favorite restaurant, every day regardless of how many choices she is

presented with. If we give her a list of 10 versus 100 restaurants, she won't even look at them, and will always choose to eat at Excellent Cuisine. We could say Alice is following a constant time algorithm, where it always takes her the same amount of time to decide on a restaurant no matter the length of the list of restaurants we provide her with. Most algorithms involve more complicated operations than those that can be done in constant time, however, so our next category of time analysis is polynomial time.

Say Bob is also choosing a restaurant, but he likes to research every restaurant on the list for 3 minutes before making a final decision. If we give him a list of 10 restaurants, it will take him 30 minutes to choose. If we give him a list of 100 restaurants, it will take him 300 minuts to choose. For any list of restaurants of length n, we know it will take him $3n$ minutes to choose. Recall that a polynomial is a mathematical expression that combines terms of variables and constants using various operations (e.g. x^4+2x^3+7 or x^2+y^2). In Bob's case, if we call his algorithm b, then we can say $b(n) = O(3n) = O(n)$ to indicate that the amount of time Bob takes to choose a restaurant scales with the length n of the list of restaurants. If Bob took 3 hours per restaurant instead of 3 minutes, we would still say $b(n) = O(3n) = O(n)$. Another way to think about polynomial time, is to consider how many actions must be completed per input to the algorithm. For instance, if I already have cookie dough and I need to bake the cookies, my algorithm might be: for every tablespoon of cookie dough, roll the dough into a ball and then sprinkle cinnamon on top. That is 2 actions to complete for every tablespoon of cookie dough. So, if we have n tablespoons, we could say this step takes $O(2n) = O(n)$ time. If I then compared each ball of dough to every other ball to make sure they were all the same size, then for each of the n balls I am comparing it to n other balls. In this case, I am performing n^2 additional actions, which we can describe as a polynomial number of actions (which is why we say it takes polynomial time). So, I can say this whole process takes $O(n^2+2n) = O(n^2)$. Because n^2 grows much faster than n, we can drop the n in Big-O notation to indicate n^2 is the relevant upper bound. For our purposes, we can consider polynomial time to be sufficiently "efficient," with the consideration that we would hope for our algorithm to be much closer to $O(n)$ time than $O(n^{100})$, as the former is much more efficient than the latter.

Our last category is exponential time, which is very slow compared to our other two categories. We consider an algorithm "inefficient" if it takes exponential time. An example of exponential time in Big-O is $O(2^n)$. As n grows larger in this case, the running time increases extremely rapidly. To compare polynomial time and exponential time, imagine we have some number $n = 1,000$. If we square n, we get a polynomial expression: $n^2 = 1,000^2 = 1,000,000$. That is indeed large, but not compared to $2^n = 2^{1,000}$, which is "a number much larger than the number of atoms in the universe" [1].

1.1.2 P vs NP

In time complexity theory, the field of mathematical analysis centered on the running time of algorithms, we give more specific names to the aforementioned categories. The most important complexity category for our purposes is P, which denotes the class of problems that can be solved with polynomial time algorithms. In other words, we can think of P as a collection of problems where every problem is solved by an algorithm that runs in polynomial time. Perhaps intuitively, we know there exist many problems that take longer than polynomial time to solve (such as those that take exponential time and therefore belong to the class Exp). A less intuitive fact is that there also exist problems that we think do not belong in P, but we don't know how to prove it at this time nor do we believe we will know how to prove it anytime soon.

To understand what I mean by this, we need to discuss another important complexity class called NP: the class of problems solved by *nondeterministic polynomial* time algorithms. In very simplified terms, one could think of "nondeterministic" as a brute force approach to difficult problems: for a problem with 2^n possible solutions, we go through all 2^n of them until we find a correct solution (assuming a correct solution exists). In the worst case, this approach will take at least 2^n actions, which we now know is not polynomial time and therefore not a feasible way to solve the problem. However, we can also think of nondeterminism to mean that if we were told which of the 2^n solutions is the correct one, we could could *verify* that it is correct in polynomial time. As an example, lets think about the classic 52-card game of solitaire [2].

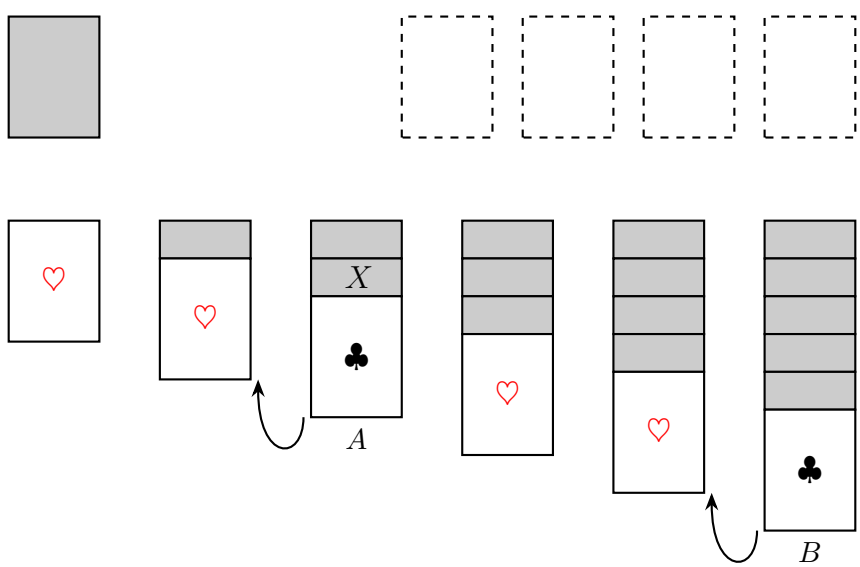


Figure 1.1: Example starting configuration of a solitaire game.

In solitaire, it is difficult to know whether a starting configuration of cards will lead to a win just by looking at it. One way we can figure out if it is a winnable

configuration is by going through every single subsequent set of moves that leads to a terminating position, and seeing if that final position is a win. For instance in Figure 1.1, we can either start by moving card A or card B. Whichever choice we make first will lead to another set of moves for each of them, and each of those moves will lead to more moves, until the game ends by either winning or getting stuck. If we perform all of the possible moves after we move card A and all of them end in a stuck position, then we still don't know whether moving card B first would lead to a win. At this point, we've done a lot of work for few results. This is an example of how the brute force method is often too difficult to carry out. However, if we set up our cards and then our friend said, "I bet if you move card A, and then card X, and then card....you will win," we can follow their instructions to verify that sequence leads to a win, therefore proving to us that the starting configuration of cards is winnable. In this case, we only had to move at most 52 cards to come to that conclusion, which means we performed a constant number of actions to solve the solitaire problem (if we say this is true for any deck of n cards, it would be a polynomial number of actions). However, unless we all have a friend that can forsee the future, we cannot guarantee that we would try the solving sequence first. So, NP is the class of problems where we can confirm a given solution is correct in polynomial time, but we don't know of a polynomial time algorithm to find that solution ourselves.

Keep in mind that problems we know are in P are also in NP, because we can verify that their solutions (which we can find in polynomial time) are correct in polynomial time. We will denote the problems that we cannot prove are or are not in P as $NP \setminus P$, which means the set difference of NP and P, or all problems in NP that are not known to be in P. Generally, we consider the class of problems in $NP \setminus P$ to be very difficult to solve, as it means no one has ever discovered an algorithm that can find a correct solution in polynomial time. The hardest NP problems are known as *NP-complete* problems, where "complete" means the problem can be generalized to take the form of every other problem in NP. So, if a polynomial time algorithm was discovered for an *NP-complete* problem, it would mean that every NP problem could be solved in polynomial time, and therefore all of those problems would actually just be in P [1].

1.2 Graph Theory

In discrete mathematics, a *graph* is a collection of nodes called *vertices* connected by *edges*. We denote $G = (V, E)$ where G is the graph, and it is composed of a set of vertices V and a set of edges E. Mathematicians like graphs because their structure makes them very effective for modeling a variety of real-world situations such as social networks (where vertices are individuals, and edges connect people to their acquaintances) or even the spread of disease (where vertices are people and edges represent an infection spreading from one person to another). Many graph problems involve finding a *path*, or a sequence of edges that joins a sequence of distinct vertices. In Figure 1.2, there are two paths from vertex a to vertex c.

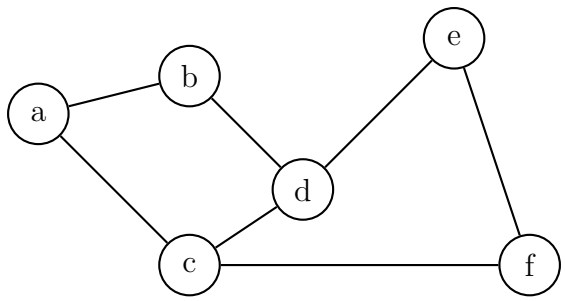


Figure 1.2: Simple graph with 6 vertices and 7 edges.

Graphs can be used to model more complicated problems by adjusting their structure slightly. A basic graph problem has vertices and edges, but we can add factors such as edge *direction* going from one vertex to another, or assigning *weights* to the edges. A "weight" in this context is some value derived from the constraints of the problem. The weighted graph can then be used to model problems where we want to maximize or minimize the weights. For instance, we can construct a graph where each vertex represents a city and each edge represents a highway from one city to another. If we were planning a roadtrip, we could add a weight to each edge to signify the travel time of that highway from one city to another. Then, if we were planning a road trip to all the cities on the graph, we could find the fastest route by finding the path with the smallest total weight. If Figure 1.3 represents our road trip, it is faster to take the edge with weight 4 from a $\rightarrow$ c than to take the path a $\rightarrow$ b $\rightarrow$ d $\rightarrow$ c, which has a total weight of 6.

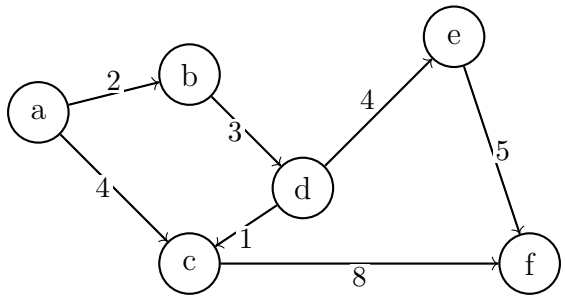


Figure 1.3: Directional graph from Figure 1.2 with weights.

There are many scenarios in which we may also want to find the maximum total weight of a graph. Many polynomial time algorithms exist for maximizing weights, one of the more well known ones being the *Hungarian* algorithm that runs in $O(V^3)$ time [3]. Algorithms such as the Hungarian algorithm can be helpful in many instances, such as assigning resources to entities. We can use edges to draw connections between resources and entities, and assign weights based on how desirable that resource is to each entity. This construction models real-life instances such as assigning taxi drivers to waiting customers, or (as we will find in this thesis) classes to students. Resource assignment problems are actually extensively studied under another field of research as well called *mechanism design*.

1.3 Mechanism Design

Mechanism design has traditionally been studied by economists and game theorists, though in recent years computer scientists have been joining the field as well because mechanisms are often just algorithms. Mechanism design is the construction of systems that "fairly" (under various definitions of the word) address scenarios involving a host of participants with adverse or overlapping goals. Some examples of problems that fall under this classification are as follows: designing a voting system that successfully eliminates strategizing by any party [4], auctioning search engine advertisement spots in a way that promotes honest bidding [5], assigning roommates from a pool of students such that each student ends up with a roommate they prefer [6], and maximizing aid while allocating food donations to food banks [7]. There are two subcategories of mechanism design relevant to this thesis: stable matching and fair division.

1.3.1 Stable Matching

Many real world scenarios can be modelled as *matching* problems, such as assigning medical students to hospitals for residency [8]. The standard (and heteronormative) matching problem is known as the Stable Marriage Problem [9], in which the goal is to match an equal number of men and women such that no participant remains uncoupled and everyone has exactly 1 partner. This is known as a *bipartite* matching problem, or one in which the market of participants is divided into two disjoint sides and matched from one side to the other. Each participant provides a list of preferences, ranking who on the other side they would like to be matched with from most to least. This is known as *two-sided* preferences, where every member of both sides provides a preference list. An essential aspect of the Stable Marriage Problem is that the matching should prevent "cheating," which occurs when a man and a woman who weren't matched to each other prefer to be together over their assigned partners and will "elope," leaving the other two participants alone. This couple would be known as a *blocking pair*, because their elopement "blocks" the matching from being complete. Herein lies the stability aspect of stable matching: we wish to find an algorithm that outputs a complete matching of couples with no blocking pairs.

In 1962, David Gale and Lloyd Shapley published a seminal stable matching paper, in which they proposed the **deferred acceptance** (DA) algorithm: a solution that guarantees a stable matching for the Stable Marriage problem in polynomial time [10]. Gale and Shapley also proved their DA algorithm could solve the College Admissions problem: matching high school student applicants to colleges. College Admissions differs slightly from Stable Marriage: it is a *many-to-one* matching in which colleges accept multiple students rather than each college matching with a single student, and it has *incomplete preferences* because students do not rank (apply to) every possible college that exists. With a slight adaptation of the DA algorithm, it can solve this version of College Admissions. Since 1962, the DA algorithm has been adapted to fit many more problems with additional factors, including the residency problem.

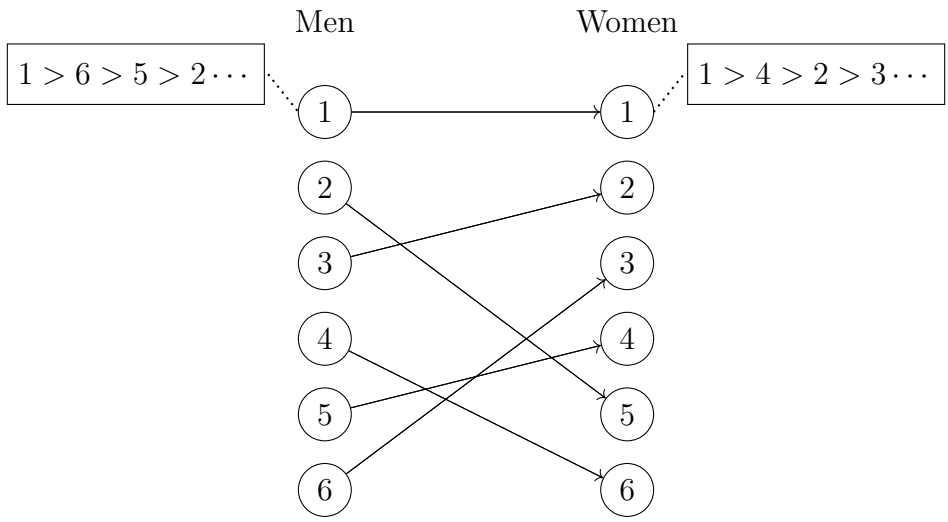


Figure 1.4: Bipartite directional graph illustrating the Stable Marriage problem.

Stable matching lays much of the groundwork for the work we do in this thesis. In a sense, we want to match students to HUM sections in a way that is "stable," though our understanding of stability will change as we continue to explore our problem because students and HUM sections cannot "cheat" on each other in the way men and women can. Despite this, much of the research that our work builds upon is based on altered versions of the DA algorithm. We will also develop a new notion of stability that expands upon this traditional definition from Stable Marriage.

1.3.2 Fair Division

The difference between matching and fair division (also known as fair allocation) is most often two-sided versus *one-sided* preferences. One-sided preferences means only one side of the bipartite market cares what they are matched to. Because only one side has preferences, we no longer have the idea that "cheating" could occur; instead, we evaluate fair division under *envy*. For instance, say we have a cake that is half chocolate and half vanilla, and we are dividing the cake between people at a party. A person who prefers vanilla but receives chocolate will be *envious* of those who receive a vanilla slice. In a fair division problem, the goal is to minimize the amount of envy between participants and maximize the amount of *welfare*, where welfare is the benefit participants yield from the goods they receive (such as feeling significantly more joy from vanilla over chocolate).

The cake-cutting example is the canon example for what is known as *divisible goods* [11], but there is also a lot of study into the allocation of *indivisible goods* where items cannot be cut up like a cake. The aforementioned food bank example, allocating donated items (such as non-perishable cans of food) to different food banks based on preferences they provide, is a fair division problem with indivisible goods [7].

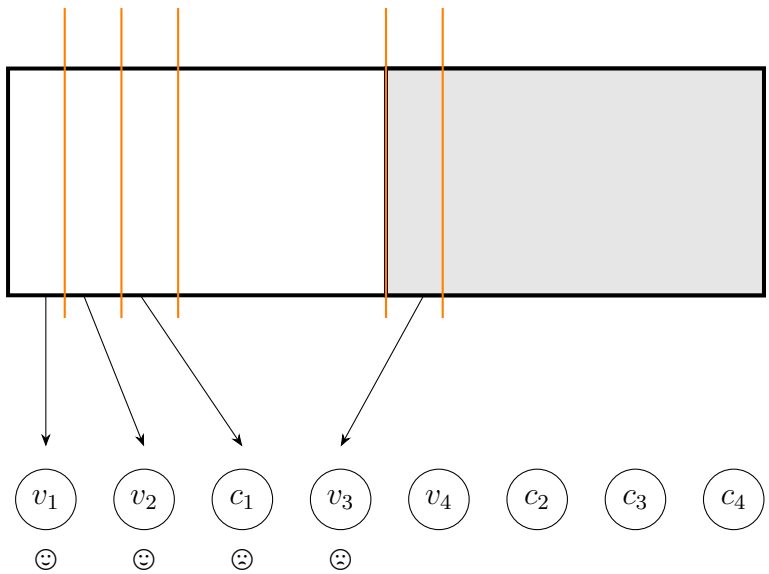


Figure 1.5: Cake cutting example where the left half of the cake is vanilla, the right half is chocolate, v_i nodes represent vanilla people, and c_i represents chocolate people. In this case, c_1 and v_3 would envy each other.

In addition to envy and welfare, fair division will often be evaluated under Pareto-optimality. An allocation is considered *Pareto-optimal* if there is no other allocation that is strictly better for at least one participant while retaining the same welfare for the other participants [12]. As more factors are added to fair allocation problems—such as considering the order in which goods arrive for distribution, scaling the number of participants, and defining more specific versions of welfare—envy-freeness and Pareto-optimality become more difficult to achieve.

Many problems like HUM section assignment are modelled as fair division problems, as they involve one-sided preferences. This includes a basic version of course allocation and an altered version of College Admissions, which we will discuss in Section 1.5. We will use some of these concepts to define our problem in Chapter 2, combining facets of stable matching and fair division.

1.4 Adding Constraints

What's known as the *vanilla* versions of these matching and division environments, which is to say the more simple or basic environments without many additional factors, have been studied thoroughly by now. So, to advance mechanism design research and to more accurately model real-world scenarios, scientists add *constraints* to the problems. Constraints come in many forms, and often make problems so complex they become *NP-complete*. In the scope of class assignment and related matching problems, constraints often come in the forms of quotas on class capacity and categories of students.

1.4.1 Quotas and Diversity

Quotas in allocation and matching problems can be implemented as upper and lower bounds on the mechanism. Upper bound quotas imply capacity constraints, such as a maximum number of open positions per hospital in the residency matching problem. Gale and Shapley's College Admissions problem, in which a college with n applicants can admit up to q students, is traditionally studied with these capacity constraints (where q is for quota). Lower bound quotas in something like the College Admissions would encode in the mechanism that schools need to accept at least m students [13]. Strict quotas can cause finding a stable matching in a bipartite setting to be *NP-hard* [14], so quotas are often studied under "soft" constraints as well: for example, allowing a quota to be a target range rather than a specific number.

The College Admissions problem can be extended to more accurately reflect reality by modelling diversity constraints. The goal of diversity constraints is usually to produce a matching/allocation in which some type of participants is spread evenly or represented accurately in the result. In the actual college admissions problem, these constraints are commonly known as Affirmative Action and have traditionally been implemented in the form of capacity quotas on majority students to give space for minority students. There are many papers that mathematically study that particular version of Affirmative Action, as well as prove that it does not always benefit the minority applicants [15]. However, diversity constraints can take many forms—they have been studied under frameworks of lower-bound quotas [13], proportionality (evenly distributing types of students in the matching) [16], and *group fairness* (equalizing outcomes across protected and non-protected groups) [17]. In each of those papers implementing diversity constraints, the constraints make the problems *NP-complete*.

1.4.2 Group Fairness

"Group fairness" in classification or assignment tasks refers to "preventing discrimination against against protected population subgroups" while still achieving some "utility" in the assignment [17]. These problems involve classifying individuals themselves by some metric of "similarity" with respect to the particular task. A lot of research has been done on defining "similarity" mathematically, but in this thesis we are more concerned with ensuring pre-determined similar students are treated fairly in class assignment. Because the "diversity" aspect of our HUM section assignment problem refers to academic subject diversity, we will use "group" to mean groups of majors (such as arts, social sciences, natural sciences, etc.). That is, we use group fairness to signify that students of different academic groups are represented proportionally within HUM sections with respect to the overall population of their group in the student body. We can also think of group fairness in our problem as the constraint that students of all academic groups at Reed have equal opportunity to be in all of the HUM sections.

1.5 Related Work

The problem we discuss in this thesis is related to two extensively-studied problems: school choice and course allocation. School choice in mechanism design is the problem of assigning high school students to high schools such that parents are satisfied, each student has a school to attend, and no school admits more students than they can accommodate. Course allocation in mechanism design is the problem of constructing a system to assign students to classes based on which classes they prefer to take.

1.5.1 Two-sided matching

In 2003, Abdulkadiroğlu and Sonmez published a foundational paper on school choice, in which they view the problem through the bipartite lens of schools and students each with distinct preferences. This lens allows them to apply the Gale-Shapley DA algorithm as a solution as well as their own algorithm, which they called Top Trading Cycles [18]. In Top Trading Cycles, if the matching has a cycle of students in which each student prefers the next student's school, each assignment is given to the next student until the cycle is broken.

Six years later, in 2009, Abdulkadiroğlu, Pathak, and Roth amended the 2003 paper with a case study into the New York City high school allocation mechanism, scrutinizing the DA algorithm under stability. They show empirically that there is a loss of Pareto efficiency in the algorithm when adding stability to the environment [19].

Many later papers extend one or both of the aforementioned papers in further research. In 2010, Biró et al. studied school choice in a college setting with both upper and lower quotas, constraining the environment in a new way. They determined that stable matchings do not always exist with those quotas, and in fact determining whether a stable matching exists at all is *NP-complete* [13]. Three years later, Hafalir et al. analyzed the DA algorithm for school choice under quotas for diversity, where they found that using upper quotas produced a matching that was Pareto-dominated by a matching with lower quotas for minority students [15]. There has since been much more research into the stability effects of quotas on the two-sided school choice problem, including papers that implicate stability with diverse students of multiple types [20].

1.5.2 One-sided matching

Since 2003, research also branched from two-sided school choice into one-sided matching, where only one side of the market has preferences. This was often studied under the framework of course allocation. In 2011, Budish and Cantillon published the defining paper on course allocation at Harvard (which cites Abdulkadiroğlu's 2003 and 2009 papers) in which they evaluate Harvard's mechanism under Pareto-optimality [21]. There have been slight alterations to their framework since then, such as the effects of incomplete preferences on participant welfare.

Only in recent years have scholars started studying one-sided matching with quotas and other diversity constraints. In fact, the fairness constraints closest to that in this paper only seem to exist in literature published as recently as March-August 2025. Specifically, Santhini et al. researched the empirical effects of both soft quotas [14] and group fairness [22] on the mechanism using convex optimization (also done similarly by Panda et al. [23]). There is not readily as much theory-based research on one-sided matching with constraints as two-sided matching, and those that do study it under a theoretical framework use more of a fair division envy/Pareto-based analysis than stability.

As far as we can tell, the closest paper to our construction of stability in this thesis is an unpublished 2011 paper by Abdulkadiroğlu, who generalizes two-sided and one-sided preferences to a single environment in which the Top Trading Cycles algorithm can be reduced to work for both kinds of preferences. In the paper, he frames stability to be "if, whenever a student prefers a school to her assignment, that school is enrolled up to its capacity by students who have higher priority at that school" [24], where "higher priority" in the one-sided setting only considers how strongly the other students prefer that school. The paper does not consider the effect of adding fairness constraints.

Chapter 2

Formalizing the Problem

The existing research on school choice and course allocation would seem to suggest that for our problem, which only features one-sided preferences as opposed to preferences on both sides of the market, we follow a fair division approach similar to that of Budish's one-sided matching [21]. However, given the practical application of our problem as a mechanism that is meant to reduce the Registrar's workload, we determined that constructing a notion of stability makes more sense than evaluating envy in a fair division manner. While we care about student happiness in our solution, our formulation of the problem is primarily driven by the goal of assigning year-long HUM sections that are balanced both in size and student majors, meaning we want to minimize the ability of students to swap with each other post-assignment so as not to disrupt the balance. If no two students recipricolly wish to swap with each other, then we can consider the assignment stable. This is the framework under which we shall formalize our model.

2.1 The Model

The basic components of our model are the two sides of our market: a set of students and a set of HUM class sections, each of which have a certain number of seats based on the class capacity. Though we are working in a bipartite setting, only one side of our market has preference over the other (i.e., students prefer certain classes, whereas classes are inanimate and therefore indifferent to students...as far as we know). Students also do not necessarily have ranked preference over every single available section; they likely equally do not want to take any sections that conflict with their other selected courses. So, preference lists will be incomplete, with those sections that students have a conflict with not being included in their ranking.

Definition 2.1. The *HUM-assignment* problem consists of a finite set of Reed College students, $\mathcal{R}$, and a finite set of class sections, $\mathcal{C}$. Each class section $C \in \mathcal{C}$ has a capacity q representing the number of seats in the class: we can think of each C as a set of seats where $|C| \leq q$. We denote the set of all seats $\mathcal{S}$, where $C \subset \mathcal{S}$ is disjoint for all $C \in \mathcal{C}$. Students rank classes in $\mathcal{C}$: each student $r \in \mathcal{R}$ has a strict preference relation $\succ_r$ over $\mathcal{C} \cup \{\emptyset\}$, where $\emptyset$ represents the outside option of a student. That is, $\emptyset \succ_r C$ indicates a student prefers no match over being matched with C for all $C \in \mathcal{C}$.

Definition 2.2. A *1-sided matching (1M)* of students to class sections is a function $\mu : \mathcal{R} \to \mathcal{C} \cup \emptyset$ such that

- Each student is assigned to a class section they prefer or no seat: $\mu(r) \succ_r \emptyset$ or $\mu(r) = \emptyset$ for all $r \in \mathcal{R}$.
- No class exceeds its capacity: $|\{r \in \mathcal{R} : \mu(r) \in C\}| \leq q$ for all $C \subset \mathcal{S}$.

Finding a 1-sided matching would solve an instance of the HUM-assignment problem as defined above. However, we still have to introduce the last constraint of the problem: *group fairness*.

Students can be divided into three groups based on the way Reed College divides subjects: Group 1 for language, arts, and literature, Group 2 for social sciences, Group 3 for math and lab sciences [25]. To implement the HUM philosophy, which is that the class provides an opportunity for students from all knowledge backgrounds and academic interests to discuss relevant texts, we want to ensure a distribution of students that "fairly" represents each group in every HUM section. Practically, this means avoiding scenarios such as all Chem 101 students with the same lab being placed in the same section due to the overwhelming similarity in their schedules. This will come in the form of *quotas or caps* on the number of seats per section that can be assigned to each group of student, meaning each class will be divided into two sets of seats. One set will be *reserved* seats, representing some number of seats in each class that are reserved for each group of student. The other set is a number of *free* seats in each class that any student can be assigned to, which will allow for some flexibility.

Definition 2.3. Under the group fairness constraint, $\mathcal{R}$ is partitioned into three disjoint sets: $X, Y, Z \subset \mathcal{R}$, where $X \cup Y \cup Z \subseteq \mathcal{R}$ and $X \cap Y \cap Z = \emptyset$. Denote students in X as $x_1, x_2, ..., x_i$, students in Y as $y_1, y_2, ..., y_j$, and students in Z as $z_1, z_2, ..., z_k$.

Definition 2.4. For each class $C \in \mathcal{C}$ with capacity q,

$$|C| = q = \sum_{t \in \{X,Y,Z,F\}} q^t$$

where $X \cup Y \cup Z \cup F \subseteq C$. Similarly to Definition 2.3, sets of seats X, Y, Z correspond to the three groups of students. We denote the set of all reserved seats $\mathcal{S}_{res} = X \cup Y \cup Z$. F represents the set of free seats. We denote a specific seat $s \in C$ reserved for group T as s_c^T.

Definition 2.5. A *group-fair 1-sided matching (F1M)* builds on Definition 2.2 of a 1M by adding the constraints that no class is assigned more students of each group than there are reserved and free seats for those student where no more than one student is matched to each free seat, maintaining that no more total seats are matched for a class than its overall capacity:

- $|\{t \in T : \mu(t) \in C\}| \leq q^T + q^F$.
- $|\{r \in \mathcal{R} : \mu(r) \in C\}| \leq q$ for all $C \subset \mathcal{S}$.
- $|\{r \in \mathcal{R} : \mu(r) \in F \subset C\}| \leq q^F$ for all $C \subset \mathcal{S}$.

To make things clearer, we can visualize our problem using a graph (see Figure 2.1).

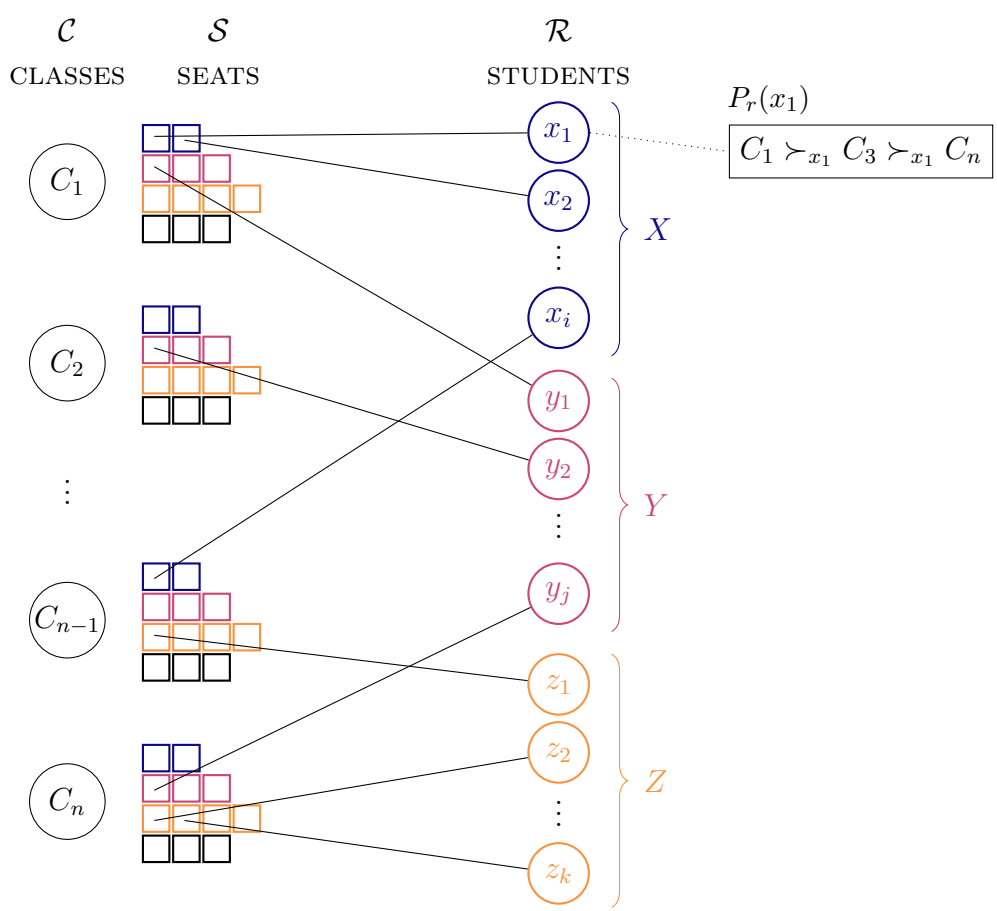


Figure 2.1: Graph illustrating an F1M, where each group of students, X, Y, and Z, is represented by a different color.

In a graph representing the HUM-assignment problem, the vertices are comprised of students and seats that belong to specific class sections, and an edge is drawn between a student and a seat if the corresponding class is on the student's preference list. We can formally define this representation.

Definition 2.6. Let $G = (V, E)$ be a bipartite graph where:

- $V = \mathcal{R} \cup \mathcal{S}$
- $E = \{(t, s_c^T) \mid t \in T \subset \mathcal{R}, s_c^T \in \mathcal{S}, C \in \mathcal{C}, C \succ_t \emptyset\}$.

In this form, the graph only represents our problem, not a solution. For a graph to represent a solution or a matching, each student would only have one edge connecting them to a single seat in a class in a way that adheres to class capacities. So, we need a way to narrow down which edges lead to a viable solution. That means we need a way to judge edges against each other: we need to assign value to them. To accomplish this, we can give each edge a weight that represents how conducive it is to the constraints of our problem based on how well that match would accomplish our goals. Recall that we want to satisfy two goals: every student is assigned to a class they prefer, and all reserved seats are filled by students of that reserved group. We call a matching *full* if every student is matched to a seat they prefer. Because these two goals are equally important to us, we can simply increment the weight of each edge by 1 for each goal it satisfies. We can now supplement our previous definition of a graph.

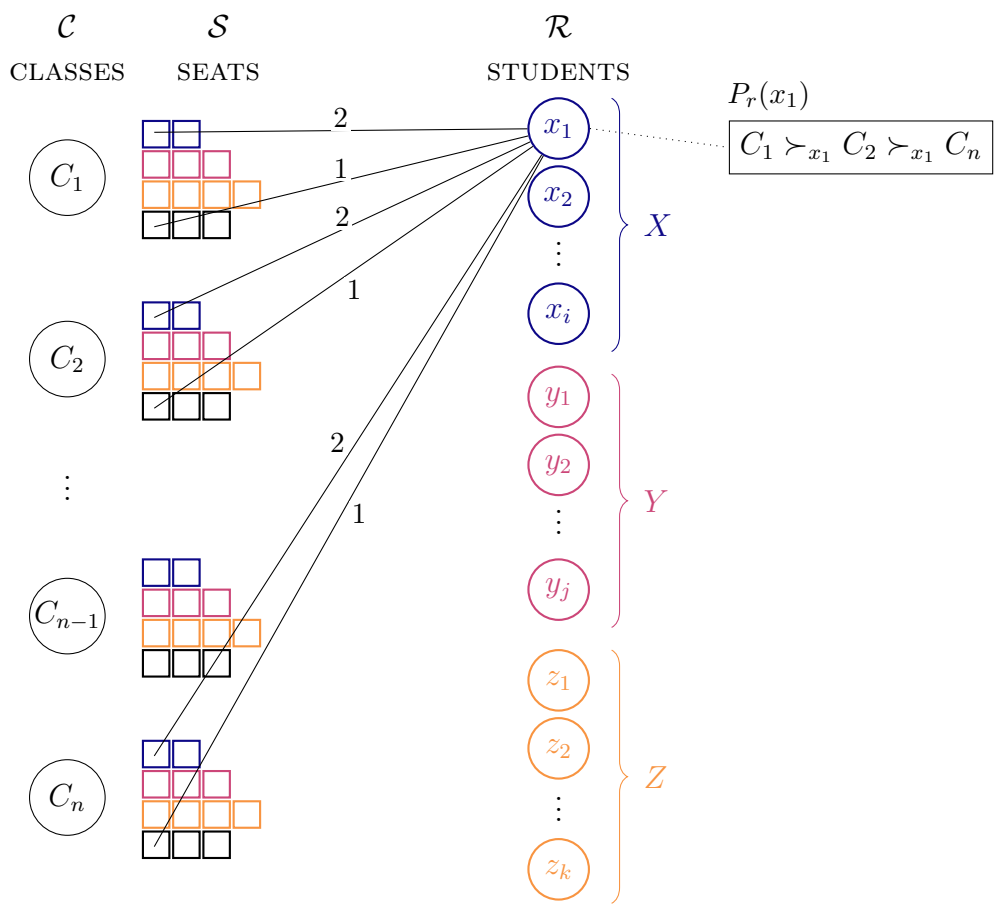


Figure 2.2: Graph illustrating weighted edges for student x_1.

Definition 2.7. Let $G = (V, E)$ be a bipartite graph where:

- $V = \mathcal{R} \cup \mathcal{S}$
- $E = \{(t, s_c^T) \mid t \in\subset \mathcal{R}, s_c^T \in \mathcal{S}, C \in \mathcal{C}, C \succ_t \emptyset\}$
- $w : E \to \{1, 2\}$ is a weight function defined as

$$w(t, s_c^T) = \mathbf{1}\{\text{if student } t \text{ prefers class } C\} + \mathbf{1}\{s_c^T \text{ is reserved for } t \in T \subset \mathcal{R}\}.$$

Now, we have a way to compare edges to each other to allow us to find a solution if one exists. We start with multiple edges between each student and various reserved and free seats in classes on their list, as shown in Figure 2.2. However, we want only one edge between each student and a seat in the class section they will be assigned to in the final matching. Based on the way we've set up our weights, we know that the "best" available solution (which satisfies the most constraints) will be the graph with the maximum total weight when only one edge remains for each student. In Figure 2.2, we can observe how maximizing the total weight works in practice: if we want to preserve just one edge for student x_1 when matching them to C_1, we would keep the edge weighted 2 to the reserved seat rather than the weight 1 edge to the free seat. Our maximized total weight matching then has the properties required by our F1M (Definition 2.5), being that students of each group are prioritized assignments to reserved seats in classes they prefer, ensuring each group is represented fairly within each class.

Definition 2.8. A matching μ is a *maximum-weight matching* if for all F1M matchings μ':

$$\sum_{r \in R} w(r, \mu(r)) \geq \sum_{r \in R} w(r, \mu'(r))$$

Observe that a maximum-weight matching would not only serve as a F1M solution, but it would also (by nature of the way we assign weights based on student preferences) ensure the solution is one in which the greatest number of students are assigned to a class on their preference list. So, viewing the problem through the lens of a weighted graph provides us with a helpful framework for satisfying the problem.

2.2 The Challenge of Stability

We've determined how to find a solution to the F1M problem from Definition 2.5, but there is one more aspect to the problem we must consider. Recall that matching is often studied under the analysis of *stability*. Notably, "stability" in our case adopts a different meaning than in a traditional stable matching problem. Because we are working with one-sided preferences, we do not consider "cheating" as a violation of stability between students and classes, but rather between students themselves. That is, we can consider a matching stable if there are no blocking pairs *between students*.

Definition 2.9. A matching μ contains a *blocking pair* between students $r_1, r_2 \in \mathcal{R}$ if $\mu(r_2) \succ_{r_1} \mu(r_1)$ and $\mu(r_1) \succ_{r_2} \mu(r_2)$.

Definition 2.10. A matching μ is *stable* if there exists no blocking pair between any two students $r_1, r_2 \in \mathcal{R}$.

This notion of stability proves difficult for our problem. If we look back at our original definition of the HUM-assignment problem (Definition 2.1) with class capacities and incomplete preferences, we will find that in some instances it is possible there may not even exist a stable matching to find.

Theorem 2.11. *There is not always a stable matching with quotas and incomplete preferences.*

Proof. We prove by counterexample that a stable matching may not exist under quota constraints and incomplete preference lists. Let $\mathcal{C} = \{C_1, C_2\}$, where there are two groups of student X and Y such that $q^X, q^Y = 1$ for each $C \in \mathcal{C}$ (each class can have at most one student of each group). Let $\mathcal{R} = \{x_1, y_1, x_2, y_2\}$. Let the preference lists be as follows:

$$\begin{aligned} x_1 &: \ C_1 \\ y_1 &: \ C_2 \\ x_2 &: \ C_1 \succ C_2 \\ x_2 &: \ C_2 \succ C_1 \end{aligned}$$

In this case, regardless of the order of matching or algorithm, there is no stable matching. The X students will both wish to be matched to C_1, which only has one X seat, and both Y students wish to be matched to C_2 which only has one Y seat. So, that means that in any matching, there is still an open Y seat in C_1 and an open X seat in C_2, which the unhappy X and Y students can then switch into on their own, respectively. □

Defining stability in this way is, then, too strict. Recall that even determining whether there is a stable matching or not is *NP-complete* in many scenarios similar to this one [14]. So, we need to relax our constraints to allow us to explore some less restrictive notion of stability. For now, let's simplify the problem. At a basic level, before we even think about stability, we want to work in an environment where we can be sure an algorithm would respect all student preferences, such that no student is assigned to a class they did not list as an option.

To temporarily remove our notion of stability, we can alter our preferences from a ranked system to what is known as *0/1 preferences*. In 0/1 preferences, students give a class a 1 if they are okay with being matched to it and a 0 if it is not an option for them. This way, there cannot be a notion of stability because all preferred classes have the same rank. Notably, this simplified environment still lends itself well to the same weighted graph construction from Definition 2.7, so we can continue to use a weighted graph to represent our problem.

Theorem 2.12. *A matching μ fills all reserved seats while only assigning students to classes they prefer if and only if $w(\mu) = |\mathcal{R}| + |\mathcal{S}_{res}|$.*

Proof. First we will prove the forward direction. Let μ be a matching that assigns every student to a seat they prefer while also filling reserved seats first, such that all reserved seats are filled before free seats. By construction of the weights from Definition 2.7, we can determine a lower and upper bound for $w(\mu)$.

1. **Lower bound of μ:** Each edge exists only if the student prefers that seat, so every edge in μ contributes at least 1. By our assumption of characteristics of μ, every student in μ got a seat they prefer, so summing over all students contributes at least $|\mathcal{R}|$ to $w(\mu)$. We also assumed each reserved seat was assigned to a student in μ, which increments the weight by $+1$ for every edge from each of those reserved seats. Summing over all reserved seats gives an additional contribution of $|\mathcal{S}_{res}|$, because the rest of the seats are still 1-weight edges. Therefore, we can bound μ:

$$w(\mu) \geq |\mathcal{R}| + |\mathcal{S}_{res}|.$$

2. **Upper bound of μ:** We know the maximum edge weight between each student and seat is at most 2. Similarly to the lower bound, we know μ has at most $|\mathcal{R}|$ edges because each student was matched to one or none of their preferences. However, because only the reserved seats contribute an additional weight to an edge, we know that the number of 2-weight edges $\leq |\mathcal{S}_{res}|$. So, the maximum summed weight of all edges must be $2|\mathcal{S}_{res}| + |\mathcal{R}| - |\mathcal{S}_{res}|$, representing the maximum number of 2-weight edges plus all students assigned to an overflow seat. Note that is equivalent to $|\mathcal{R}| + |\mathcal{S}_{res}|$. Therefore, we can bound μ:

$$w(\mu) \leq |\mathcal{R}| + |\mathcal{S}_{res}|.$$

Note that any possible matching must then have a total weight $w \leq |\mathcal{R}| + |\mathcal{S}_{res}|$. Thus, if $w(\mu) = |\mathcal{R}| + |\mathcal{S}_{res}|$, μ is a maximum-weight matching with a lower bound of $|\mathcal{R}| + |\mathcal{S}_{res}|$ and an upper bound of $|\mathcal{R}| + |\mathcal{S}_{res}|$. Therefore, it must be that a matching that fully respects student preferences and seat reservations has a weight $w(\mu) = |\mathcal{R}| + |\mathcal{S}_{res}|$.

Now, we prove the backwards direction. Let μ be a matching where $w(\mu) = |\mathcal{R}| + |\mathcal{S}_{res}|$. Observe that $|E| \leq \mathcal{R}$, as $\mathcal{R}$ makes up one side of the market, so there can be at most $\mathcal{R}$ matches. The number of 2-weight edges then must be greater than or equal to $|\mathcal{S}_{res}|$, because $w(\mu) - |\mathcal{R}| \leq |\mathcal{R}| + |\mathcal{S}_{res}| - |\mathcal{R}| = |\mathcal{S}_{res}|$. However, we know that there are at most $|\mathcal{S}_{res}|$ possible 2-weight edges, so in fact if the number of seats matched is greater than and less than $|\mathcal{S}_{res}|$, then we know exactly $|\mathcal{S}_{res}|$ seats were matched. Thus, a maximum-weight matching where $w(\mu) = |\mathcal{R}| + |\mathcal{S}_{res}|$ is a matching that fills all reserved seats while only assigning students to classes they prefer. □

Theorem 2.12 shows us that we can identify a way to assign each student to a class they prefer while respecting group fairness constraints if such a matching exists.

From here, we can begin to reintroduce stability. While it may not be feasible to find stability between every student, perhaps we can start by comparing just students of the same group. We are introducing a new notion of stability here, where we will consider a matching stable if it exhibits *intra-group stability*. This means that for a student of any group, that student does not form a blocking pair with another student *of the same group*.

Definition 2.13. A matching μ is *intra-group stable* if, for any two students t_1 and t_2 for all groups $T \subset \mathcal{R}$, $\mu(t_1) \succ_{t_1} \mu(t_2)$ or $\mu(t_2) \succ_{t_2} \mu(t_1)$. In other words, there exist no intra-group blocking pairs.

With our new goal of intra-group stability, we can now add to our understanding of a viable solution to our HUM-assignment problem.

Definition 2.14. An *intra-group stable, group-fair 1-sided matching (ISF1M)* is a function $\mu : \mathcal{R} \to \mathcal{C}$ such that

- Each student is assigned to a class they prefer (the matching is full): $\mu(r) \succ_r \emptyset$ for all $r \in \mathcal{R}$.
- No class exceeds its capacity: $|\{r \in \mathcal{R} : \mu(r) \in C\}| \leq q$ for all $C \subset \mathcal{S}$.
- No class exceeds its capacity for each student group and free seats: $|\{t \in T : \mu(t) \in C\}| \leq q^T + q^F$ and $|\{r \in \mathcal{R} : \mu(r) \in F \subset C\}| \leq q^F$ for all $C \subset \mathcal{S}$.
- There exist no intra-group blocking pairs: for all $t_1, t_2 \in T \subset \mathcal{R}$, $\mu(t_1) \succ_{t_1} \mu(t_2)$ or $\mu(t_2) \succ_{t_2} \mu(t_1)$

For now, our goal will be to develop an algorithm that finds an ISF1M to solve our problem. We will classify our previous definition of stability (Definition 2.10) as *inter-group stability* (blocking pairs involving students of different groups) to draw a distinction between the two notions. For now, we will consider an ISF1M to be a successful solution to the HUM-assignment problem even though it is not inter-group stable. Though we would ideally guarantee stability between all students, looking at the problem through the lens of intra-group stability allows us to give some theoretical guarantee about minimizing the amount of swapping that will occur in an assignment while still remaining computationally feasible, as opposed to turning to less-reliable heuristic solutions in an attempt to solve the broader conception of the problem.

Chapter 3

Developing an Algorithm

Now that we've formally defined our problem and ideal solution, we need to actually create a way to find that solution. We also need to keep in mind that any algorithm we develop must improve upon the current system, as there already exists a standard method for HUM 110 sign-ups. So, we will evaluate the current system against our newfound ISF1M goals and determine if we can develop an algorithm that accomplishes those goals more effectively.

3.1 First Come, First Served

As it currently stands, the sign-up process for HUM 110 classes takes place during freshman class registration. The registration software, SOLAR, opens to incoming freshmen at a certain time, and at that time (or later) all freshmen log on to the platform and choose their own academic year schedule in whatever order they desire. This includes their HUM 110 section (or, as is the driving force behind this project, sections plural). Though this process may not inherently seem like an algorithm, it can actually be modelled as the algorithm commonly known as *First Come First Served* (FCFS).

FCFS takes some ordering of students that represents the registration order (in this case, those who sign up for HUM 110 the fastest to slowest after the registration software opens), and at their point of selection each student chooses their top available choice until every student has chosen a HUM class. In this case, "available" means classes that have not yet reached capacity. We write out FCFS formally in Algorithm 1, where P_r represents the ordered preference list for a student r (corresponding to their preference relation defined in Definition 2.1).

There are various benefits to a FCFS algorithm, one of the most significant for our purposes being that the given matching will be *inter-group stable*. Because each student chooses their most preferred option when it is their turn to select, there can be no two pairs of students that wish to switch. Theorem 3.3 proves FCFS stability later in this chapter. Additionally, in practice many students get their first or second

Algorithm 1 First Come First Served

```
for each student r in R do                          ▷ where R is an ordered set
    for each class C in P_r do
        if # students assigned to C < q then        ▷ class is not at capacity
            μ(r) ← C
            break                                   ▷ move to next student
        end if
    end for
end for
return matching μ
```

choice of HUM section under FCFS, and the algorithm is easy to implement because it is essentially already in use.

However, FCFS has other issues. For instance, it is possible some students will not get to choose a class they want at all if their preferred sections are already at capacity. Self-selection also allows for inconsistent class sizes and it does not consider student groups, thus permitting an inconsistent distribution of students. Our goal in this section is to develop an algorithm that improves upon FCFS with group fairness.

3.2 A Greedy Approach

We introduce group fairness into our algorithm with what we call a *greedy* algorithm: one that attempts to maximize the rank a student gives their assignment while following the group fairness constraints. We call it *greedy* because for each student, we still assign them their best available choice at the time of evaluation. However, in addition to the class capacity constraint, "available" also considers the group caps on the seats from Definition 2.4. So, a class that is not yet at capacity is still unavailable to a student of group T if it has already reached its maximum number of group T students. Essentially, GREEDY implements FCFS under the additional group fairness restriction.

Algorithm 2 Greedy

```
for each student t in R do          ▷ where t is a student of some group T ⊂ R
    for each class C in P_t do
        if # students assigned to C < q and # T students assigned to C < q^T
    then
            μ(t) ← C
            break                                   ▷ move to next student
        end if
    end for
end for
return matching μ
```

Let's compare GREEDY to FCFS. The primary difference is in line 3, where we add the group cap constraint as part of the conditional check for whether a class is available. Recall from Chapter 1 that a "vanilla" environment is the most basic version of a problem; in our case, a vanilla environment removes group fairness constraints, simplifing the problem to treat all students the same when assigning class sections. We can start by showing that in a vanilla environment, both algorithms output a stable matching.

Lemma 3.1. FCFS *is equivalent to* GREEDY *in a vanilla environment.*

Proof. Observe that in a vanilla environment, $q^T = q$ because there are no group caps. So, the second part of the `and` statement in line 3 of Algorithm 2 will always be true when the first part of the statement is true. Therefore, both algorithms will behave equivalently on the same ordered input of students in the first `for` loop, as the conditional statements check the same condition as line 3 of Algorithm 1. Thus, FCFS is equivalent to GREEDY in a vanilla environment. □

Lemma 3.2. GREEDY *outputs an inter-group stable matching in a vanilla environment.*

Proof. Let μ denote the outputted matching over an ordered set of students, $\mathcal{R}$. Let $r_1, r_2 \in \mathcal{R}$, where r_1 comes before r_2 in the order. Assume for the sake of a contradiction that r_1 and r_2 form a blocking pair, meaning $\mu(r_2) >_{r_1} \mu(r_1)$ and $\mu(r_1) >_{r_2} \mu(r_2)$.

By line 3 of Algorithm 2, we know $\mu(r_1)$ was either r_1's top available choice or r_1 is unmatched. So, if $\mu(r_2) >_{r_1} \mu(r_1)$, then $\mu(r_2)$ must not be available when r_1 is matched, which means r_2 was matched first. However, this is a contradiction of our assumption that r_1 comes before r_2 in the input to the algorithm. Thus, $\mu(r_2)$ is available when r_1 is matched, so it must be the case that $\mu(r_1) >_{r_1} \mu(r_2)$. Therefore, r_1 and r_2 do not form a blocking pair, and we can conclude that μ is stable. □

Theorem 3.3. FCFS *outputs an inter-group stable matching.*

Proof. By Lemma 3.1, we know FCFS is equivalent to GREEDY in a vanilla environment. Lemma 3.2 proves that in a vanilla environment, GREEDY outputs an inter-group stable matching. So, together those two lemmas imply that FCFS outputs an inter-group stable matching. □

Now, we can formally evaluate what happens when we do consider groups. As we discussed in Theorem 2.11, we know an inter-group stable matching may not exist at all when we introduce caps on groups. However, even assuming there does exist an inter-group stable matching, GREEDY is still not guaranteed to find it.

Theorem 3.4. GREEDY *does not always output a inter-group stable matching in a general environment.*

Proof. Let $\mathcal{R} = X \cup Y$ where there are two groups of students, group X and group Y. Let $\mathcal{C} = \{C_1, C_2, C_3 \mid q^X = 1, q^Y = 1\}$, meaning each $C \in \mathcal{C}$ can have at most one student from each group. Let $\mathcal{R} = \{x_1, y_1, x_2, y_2\}$ be ordered where students $x_1, x_2 \in X$ and $y_1, y_2 \in Y$. Let the preference lists of the students be as follows:

$$\begin{aligned} x_1 &: \ C_1 \succ C_3 \\ y_1 &: \ C_2 \succ C_3 \\ x_2 &: \ C_1 \succ C_2 \\ y_2 &: \ C_2 \succ C_1. \end{aligned}$$

Observe that an inter-group stable matching μ' does exist in this environment:

$$\begin{aligned} x_1 &\leftarrow C_1 \\ y_1 &\leftarrow C_3 \\ x_2 &\leftarrow C_2 \\ y_2 &\leftarrow C_2. \end{aligned}$$

We know μ' is stable because x_1 and y_2 both received their top choice, so neither of them wish to swap with another student. y_1 wishes to swap with x_2, but $\mu'(x_2) \succ_{x_2} \mu'(y_1)$, so they do not form a blocking pair. Similar logic follows for y_2, who y_1 would also want to swap with. x_2 wants to swap with x_1, who we already know does not wish to swap with anyone. Therefore, μ' does not have any blocking pairs, and thus it is inter-group stable.

Now let μ denote GREEDY's outputted matching over a set of students, $\mathcal{R}$. We will prove by counter example that μ is not stable. Observe that $\mu(x_1) = s^X_{c_1}$ and $\mu(y_1) = s^Y_{c_2}$: because they are the first two students to be matched and their first choices are different, they both receive their top choices. Next, $\mu(x_2) = s^X_{c_2}$ because C_1 already has a student in group X, so C_1 is no longer available when matching x_2, and similarly $\mu(y_2) = s^Y_{c_1}$ because C_2 already has a student in group Y. Notice that $\mu(y_2) >_{x_2} \mu(x_2)$ and $\mu(x_2) >_{y_2} \mu(y_2)$, which forms a blocking pair where x_2 and y_2 are incentivized to switch classes with each other. So, it must be the case that μ is not an inter-group stable matching and therefore GREEDY cannot guarantee an inter-group stable matching under group cap constraints, even when an inter-group stable matching exists. □

Notably, the above counterexample does exhibit intra-group stability as $\mu(x_1)$ is x_1's top choice, so they will never want to swap with x_2 (and similar for y_1 and y_2). In fact, we claim that GREEDY will always output an intra-group stable matching. At each point of evaluation for a student of some group, they are given their highest ranked available seat for that group. So within groups, we have a similar stability argument to Lemma 3.2.

While GREEDY is intra-group stable, its primary issue is that it does not guarantee a full matching. Students can be left unmatched if every class section on their preference list is already at capacity for their gorup. We need to come up with an algorithm that makes use of our helpful graph framework from Definition 2.7 if we want to get closer to creating ISF1M solutions.

3.3 The Weight of the Situation

Let's return to our conception of the HUM-assignment problem as modeled by a graph. We know by Theorem 2.12 we can use a maximum-weight matching algorithm to find a solution that is full and group fair. Notably, this is only one requirement away from being an ISF1M solution (see Definition 2.14): we just need to reintroduce intra-group stability to our max-weight matching. To do that, we need to remove all blocking pairs between students of the same group while preserving the weight of the max-weight matching, as the weight is how we determined that the matching is full and group fair. Here we define Algorithm 3, called the *Weighted-Trading* (WT) algorithm.

Algorithm 3 Weighted-Trading

1: $w(\mu) \leftarrow 0$
2: **while** $w(\mu) \neq |\mathcal{R}| + |\mathcal{S}_{res}|$ and # reserved seats for all groups > 0 **do**
3: $\quad \mu \leftarrow$ MAX-WEIGHT$(\mathcal{R}, \mathcal{S})$
4: $\quad$ **if** $w(\mu) \neq |\mathcal{R}| + |\mathcal{S}_{res}|$ **then**
5: $\quad\quad$ **for** each group of seats $T \subset \mathcal{S}$ **do**
6: $\quad\quad\quad |T| = |T| - 1$ $\quad\quad$ ▷ reduce the size of $\mathcal{S}_{res}$
7: $\quad\quad$ **end for**
8: $\quad$ **end if**
9: **end while**
10: **for** each group $T \subset \mathcal{R}$ **do**
11: $\quad$ **while** $\exists$ blocking pair t_i, t_j **do**
12: $\quad\quad \mu(t_i) \leftarrow \mu(t_j)$, $\mu(t_j) \leftarrow \mu(t_i)$
13: $\quad$ **end while**
14: **end for**
15: **return** matching μ

We can break the WT algorithm into two key stages: lines 1 to 9 and lines 10 to 15.

The first stage is the *weighted* part of the algorithm: it repeatedly runs MAX-WEIGHT until it finds a satisfying assignment. If the weight of the first MAX-WEIGHT run is not $|\mathcal{R}|+|\mathcal{S}_{res}|$ (as per Theorem 2.12), we "relax" our seat reservations to allow for more students of various groups to fill the class. This involves decrementing the number of reserved seats per group by one and then re-running MAX-WEIGHT now that the set of reserved seats is smaller. If the seat reservations are relaxed to the point

that there are no more reserved seats, it means there is no group-fair solution and we simply use the MAX-WEIGHT output to get a full matching for students without considering group caps. Then, we still proceed with the trading stage for intra-group stability.

The latter stage is the *trading* part of the algorithm, which handles our intra-group stability requirement. We check for all existing blocking pairs between students of the same group, and we swap or trade their assignments with each other. Notably, we have to continually check each pair in a group of students until no more blocking pairs remain, as checking each student once does not account for new blocking pairs that may arise from swapping the assignments of two students. Example 3.5 demonstrates this complication. At the end of the algorithm, we have a full, group fair, intra-group stable matching (or an ISF1M) where each student is assigned a class they prefer if such a matching exists.

Example 3.5. Let $x_1, x_2, x_3 \in X \subset \mathcal{R}$ and $\mathcal{C} = \{C_1, C_2, C_3\}$. Let the preference lists of the students be as follows:

$$\begin{aligned} x_1 &: \ C_1 \succ C_2 \succ C_3 \\ x_2 &: \ C_3 \succ C_2 \\ x_3 &: \ C_2 \succ C_1. \end{aligned}$$

Let the μ be the max-weight matching output at line 9 of Algorithm 3 where

$$\begin{aligned} \mu(x_1) &= C_3 \\ \mu(x_2) &= C_2 \\ \mu(x_3) &= C_1. \end{aligned}$$

In this case, $\mu(x_2) >_{x_1} \mu(x_1)$ and $\mu(x_1) >_{x_2} \mu(x_2)$, where x_1 and x_2 form the only blocking pair. In the trading stage, we swap their assignments so that $\mu(x_1) = C_2$ and $\mu(x_2) = C_3$. Now, observe that x_1 and x_3 form a new blocking pair, though they didn't before. When we make the swap between x_1 and x_2, it causes $\mu(x_1) >_{x_3} \mu(x_3)$, while $\mu(x_3) >_{x_1} \mu(x_1)$ was already true. So, when performing trading, we have to continually check blocking pairs for each student because new ones may arise during the process.

We can assert that the outputted matching after the trading stage is still full and group fair by proving that the weight of the graph is still the same maximum weight found in the weighted stage of WT.

Theorem 3.6. *For a matching μ output by the* WT *algorithm, $w(\mu) = |\mathcal{R}| + |\mathcal{S}_{res}|$ if such a matching exists.*

Proof. At line 9 of Algorithm 3, $w(\mu) = |\mathcal{R}| + |\mathcal{S}_{res}|$ because it was just output by MAX-WEIGHT, assuming such a matching exists. So, we have to evaluate the `for` loop that compares each student of one group to each other student of that same group. In the `for` loop, the students swap seats if they both prefer each other's match in μ. This means that each student still prefers their new match, so the weight of the

edge retains weight at least 1 from that student. Then, if either student is in a seat reserved for that group, we replace them with a student of the same group, so the edge retains the additional weight of the reserved seat. Therefore, swapping any two students of the same group does not change the weight of μ. □

While it would be nice to guarantee inter-group stability, recall that the only known algorithms for assignment problems that consider inter-group stability and group fairness are *NP-complete* [14]. With the WT algorithm, we can guarantee that if an ISF1M exists, we can find it in an efficient amount of time.

Theorem 3.7. *The* WT *algorithm will output an intra-group stable matching in cubic time with respect to the vertices in G.*

Proof. Let μ be the matching output by MAX-WEIGHT at line 9 of Algorithm 3. Observe lines 11 and 12, which outline that within each student group, we must swap the assignments of blocking pairs until no more remain. We know this loop will terminate because of our assumption that preference lists are finite, so even in the worst case students will eventually reach their highest possible preference and no longer be able to form blocking pairs. The maximum length of a preference list is equivalent to the number of classes, $|\mathcal{C}|$. So, the maximum number of times the `while` loop can execute per `for` loop is as follows:

$$|\mathcal{C}| \times |\mathcal{R}|.$$

This is because at most we can traverse each student's entire preference list once. If we terminate this loop for each group of student $T \subset \mathcal{R}$, then we must have intra-group stability because no more blocking pairs remain in each group.

We can analyze the runtime of this algorithm. Since the `while` loop executes at most $|\mathcal{C}| \times |\mathcal{R}| \times \#$ groups times, we can bound the execution of the `while` loop to $O(|\mathcal{C}| \times |\mathcal{R}| \times \#$ groups$)$. Notably, $|\mathcal{C}| \leq |\mathcal{R}|$ and $\#$ groups $\leq |\mathcal{R}|$. In practical cases there will be significantly fewer student groups and class sections than there are students, so the significant factor for this bound is $|\mathcal{R}|$. Therefore, the trading stage of the algorithm will run with respect to the size of $\mathcal{R}$.

Observe that before that, we run MAX-WEIGHT at least once, which runs in $O(V^3)$ time where V is the set of vertices [3]. Our vertices are all the students and seats, so the runtime for the weighted stage is $(|\mathcal{S}| + |\mathcal{R}|)^3$. So, our total runtime for the algorithm is $O((|\mathcal{S}| + |\mathcal{R}|)^3 + (|\mathcal{C}| \times |\mathcal{R}| \times \#$ groups$)) = \mathrm{O}((|\mathcal{S}| + |\mathcal{R}|)^3)$. Therefore, the WT algorithm outputs an intra-group stable matching in cubic time with respect to the vertices in G. □

By Theorem 3.7, we now have an algorithm that theoretically gives us a group-fair, intra-group stable matching. Note that the matching is also full, as every student is matched to a class in the MAX-WEIGHT section if such a matching is possible. Thus, the WT algorithm we've developed outputs an ISF1M solution to the HUM-assignment problem when one exists. The next step is to evaluate it against FCFS and GREEDY to determine whether it performs well in practice.

Chapter 4

Experiments

4.1 Data

In collaboration with the Reed College Registrar, we obtained three years of anonymized student and class schedule data for the purpose of evaluating our algorithms. The student data compiles the fall and spring class schedules chosen by every student registered as a freshman for that year, and is de-identified by representing each student with a randomly generated ID number. We use this information to ensure students are not matched to a section with a time conflict. The data also includes the students' top prospective major from their application to Reed, which we use for sorting students into groups. While the prospective major may not accurately reflect each student's chosen major by the time they are in class, it provides a good baseline for what discipline a student is interested in, which is sufficient for our group fairness purposes. It also may be the only information available anyway for determining groups in an actual implementation of the algorithm. The Registrar also provided us with the master class schedules for each year corresponding to the student data, so we can match students to the actual sections that were available for their year.

4.2 Implementation

To perform experiments, we implemented each algorithm in Python. We chose Python for the fact that it is a popular higher-level programming language, which means it will be easier for others to read and understand should this algorithm actually be implemented after this thesis is complete. It is also an object-oriented language, which means we could represent the components of our model as Python classes with properties, such as a `Student` class, which holds each student's provided data, and a `Class` class, which can hold information such as capacity, enrolled students, and meeting times. The object-oriented approach helps the implementation more intuitively represent our actual scenario. Additionally, Python has a very accessible csv library that allows us to parse our data into these objects. While Python may not

be as fast as other languages with some similar features, we decided that because the WT algorithm performs in polynomial time (see Theorem 3.7) and Reed has relatively small class sizes, the impact on the performance of the algorithms was less essential than the readability of the code.

4.2.1 Synthesizing Preferences

While we did have access to the student schedule and major data, we were still missing a key component of a matching problem: rankings. Because there was no feasible way of acquiring each student's actual preferences from class registration years ago, we needed to synthesize student rankings over the HUM sections to effectively simulate our HUM-assignment environment.

According to conversations with The Registrar throughout this process, Reed College students are generally less inclined to wake up early and take longer classes—which means that 50-minute classes held later than 10am tend to be the most popular choices. To simulate this preference distribution, we used an implementation of the Mallows Ranking Model [26] from the Python library *PrefSampling* [27]. This model is a popular way of generating ranking data for simulating elections, and is one of the more effective methods of doing so [28]. The MALLOWS model takes a "central vote" representing the most popular real-life choices and deviates the outputted rankings from this according to some dispersion coefficient $\phi = [0, 1]$. A ϕ closer to 1 means rankings that differ significantly from the central vote are more likely to be generated. We found an effective ϕ value for our purposes was 0.4, which allows for some deviation but overall keeps the rankings more central. In practice, this manifests in the form of most students having a 50-min MWF section as their top choice, though there are approximately 35% of students who have a longer TTh class as their top choice.

In `synthesize_preferences`, we iterate through each student's schedule and find all of the HUM sections that do not conflict with their other chosen classes (including the HUM they actually chose that year). Then, we turn those available sections into a central vote by weighting them by their time and class period length, where later and shorter sections are weighted higher. We run that vote through MALLOWS to generate the student's preferences over every section that doesn't have a time conflict. On average, there were 24 total HUM 110 sections in each year of the data sample, so a synthesized preference list could be close to 24 sections long. In order to simulate other unknown time conflicts, as well as the fact that students may not have preference over every individual section, we truncate the lists to a maximum length of 5 or 10 sections depending on the trial, or we maintain the full length lists. A relevant consideration to note here is that because our student schedule data is from years where students had to conciously make room on their schedule for their desired HUM sections, it is possible there is a bias over which sections do not have a time conflict. For instance, if multiple HUM sections were scheduled at 12-12:50pm, more students had to create the rest of their schedule in a way that didn't include other classes scheduled at 12pm that they may have been otherwise interested in taking.

4.2.2 Notes on the Algorithms

FCFS and GREEDY both involve an ordering of students; in real-life, this is in the form of whichever students are fastest to sign up for HUM. In our implementation, we randomly shuffle the list of students and then iterate through to represent this ordering. Note that this means we do not test the algorithms against an adversarial ordering for FCFS or GREEDY, and that both of those algorithms could actually benefit from the random ordering. This does not apply to WT, as WT does not use an ordering at all.

For the WT algorithm, we represent the problem as a weighted graph of students to seats (recall Figure 2.2). Unfortunately, our computer cannot read a visualization of the graph in the same way a human can. So, we need to represent our graph in code in a way the computer can work with. We use what is known as an *adjacency matrix*, or more specifically, a *biadjacency matrix*. We use a biadjacency matrix because of our bipartite setting, where the rows of the matrix represent students and the columns represent seats in each section. The matrix is then of size $|\mathcal{R}| \times |\mathcal{S}|$. At each index we calculate the weight of the edge between the student and that seat in accordance with Definition 2.7. For instance, in Figure 4.1, student 0 has weight 0 for seat 0, so we place a 0 at 0,0 in the matrix. A 0 means there is no edge between that student and seat. Similarly, student 0 has weight 2 for seat 1, so we have a 2 at 0,1.

$$\begin{bmatrix} 0 & 2 & 0 \\ 1 & 1 & 1 \\ 0 & 0 & 1 \end{bmatrix}$$

Figure 4.1: Biadjacency matrix representing 3 students and 3 seats.

The biadjacency matrix we generate is the input to MAX-WEIGHT. In Section 3.3, we cite the Hungarian algorithm [3] as a specific MAX-WEIGHT algorithm for computing the runtime in Theorem 3.7. In practice, we actually use an algorithm called *LAPJVsp*, implemented in the SciPy library [29]. LAPJVsp also performs in $O(n^3)$ time theoretically, but in practice is usually much faster [30], which is helpful when repeatedly running trials of the algorithm in our experiments.

The matching returned for all algorithms is a Python dictionary with student IDs as the keys, and the ID of the class they were assigned to as values.

4.3 Results

Because there are elements of randomness in our experimental setup, as well as multiple variables that effect the output of the algorithms, we ran many trials to accurately evaluate the performance of each algorithm. Specifically, there is randomness involved in MALLOWS, meaning MALLOWS does not necessarily output the same preferences

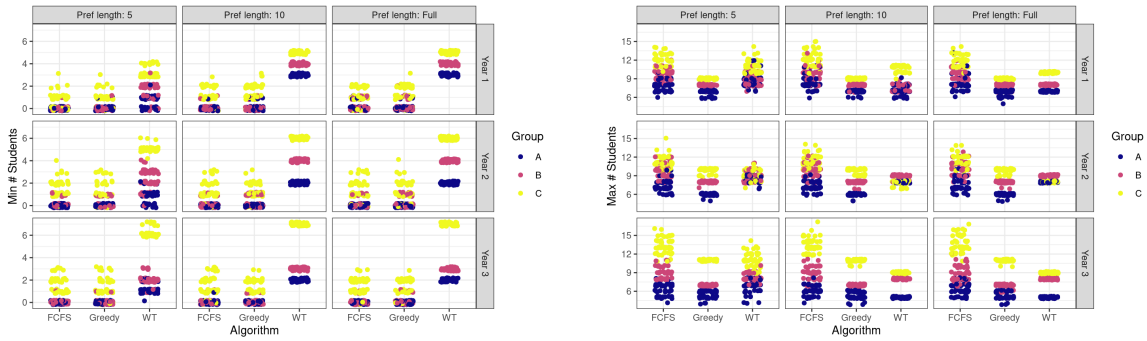
each time it is called, as well as the randomness in the ordering of FCFS and GREEDY. Additionally, the length of the preference lists is another variable that affects whether students receive a class they prefer.

We ran 50 trials each on preference lists of lengths at most 5, 10, and full (not truncated) for all three years of data per algorithm. We want to analyze the results against the following metrics in each trial, chosen based on our ISF1M construction (Definition 2.14) and our original motivation for the project:

- How well distributed the groups of students are among classes
- The number of blocking pairs in the final matching
- The number of unmatched students
- The average rank a student gives their assignment to indicate student satisfaction
- How balanced sections are in terms of class size

4.3.1 Is it fair?

Firstly, we can measure how well distributed the groups of students are by looking at the minimums and maximums for each trial. That is, what is the minimum number of students of each group in each section, and what is the maximum? These metrics will indicate to us how group fair the matchings output by each algorithm are.



(a) (b)

Figure 4.2: (a) Comparison of minimum number of students of each group over classes in all trials. (b) Comparison of maximum number of students of each group over classes in all trials.

Figures 4.2a and 4.2b illustrate how students are distributed within groups. Notably, in each sampled year, the set of all students was approximately composed of 20% group A, 30% group B, and 50% group C. This distribution of students is reflected in the minimums and maximums. For instance, in Figure 4.2b, FCFS did not proportionally assign students of group C as successfully as WT or GREEDY in comparison to the other groups over all of the years and preference lengths, as there is

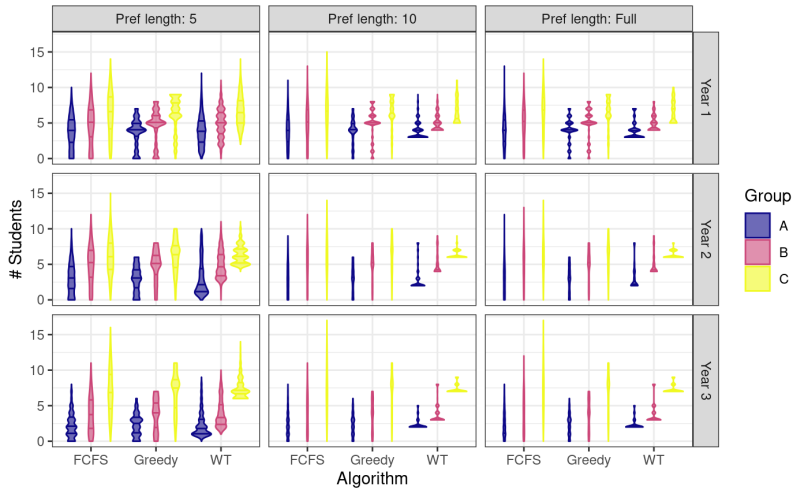


Figure 4.3: Comparison between number of students by group assigned to each class over varying preference list lengths and years of sample data.

a noticeably higher maximum number of group C students shown in each plot. It is also important that in Figure 4.2a, FCFS and GREEDY frequently distributed students such that almost every trial had sections with 0 students of some group, whereas in all of the WT trials with preference lists longer than 5 there was some representation of each group in every class.

We also visualize the overall distribution of groups amidst classes in Figure 4.3, which shows that WT creates a much less variable distribution of students. All of the classes, especially in trials with longer preference lists, have a smaller difference between the minimum and maximum number of students of each group than FCFS and GREEDY. Thus, WT accomplishes group fairness much more successfully than the other two algorithms.

4.3.2 Is it stable?

Another metric we are interested in is our original conception of stability, which means we want to check how many inter-group blocking pairs are present in the final matching output by each algorithm. Figure 4.4 illustrates that there are 0 blocking pairs in any matching output by FCFS, which guaranteed by the results of Theorem 3.3. We can also observe that GREEDY and WT both have blocking pairs with WT having significantly more, which is also consistent with Theorem 3.4 and our problem framework for WT in Definition 2.14 that disregards the presence of inter-group instability.

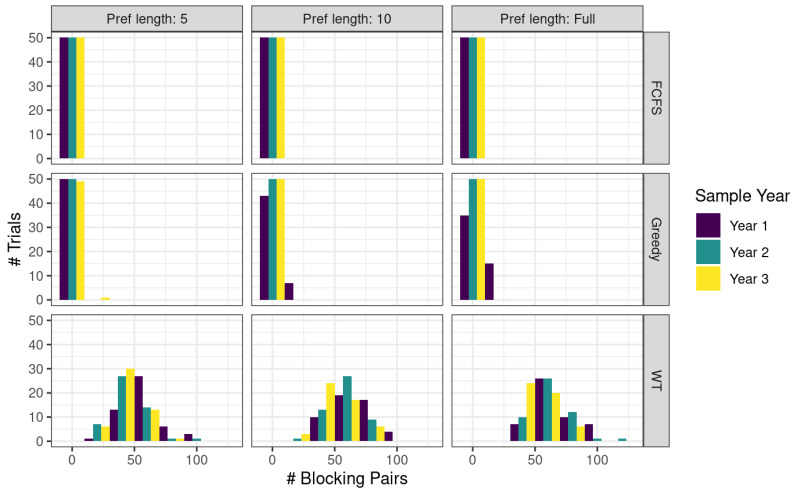


Figure 4.4: Comparison of inter-group blocking pairs present in each trial.

Relatedly, we check how many students remain unmatched in the final matching in each trial. While WT may produce the most blocking pairs as seen in Figure 4.4, Figure 4.5 shows that it does successfully match every student to a year-long HUM section in all trials. We can also observe that FCFS and GREEDY always have unmatched students with preference list length less than 10, and tend have at least 5 unmatched students per match.

We can see the effects of fairness and stability relate to the average rank a student gives their assignment. We calculate average rank with 0 being a student's top choice, 1 being their second choice, 2 being their third choice, and so on until the end of the list. If a student is unmatched, we give them a rank one more than the length of the preference list (ex. rank 6 for lists of length 5).

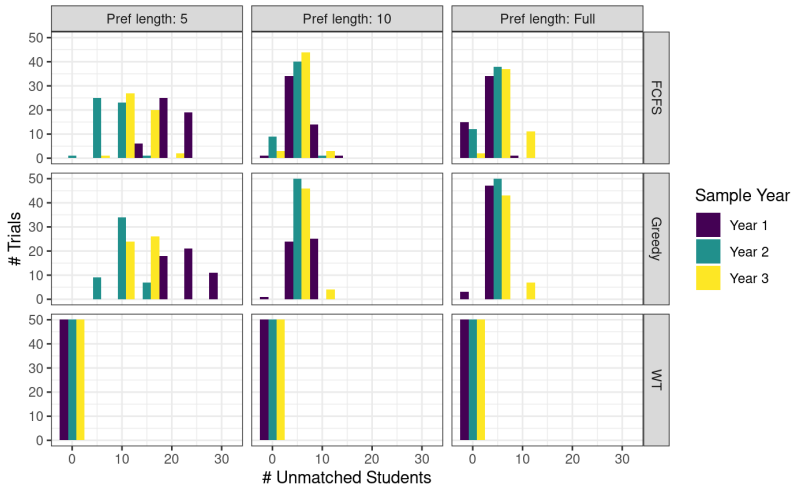


Figure 4.5: Comparison of number of unmatched students in each trial.

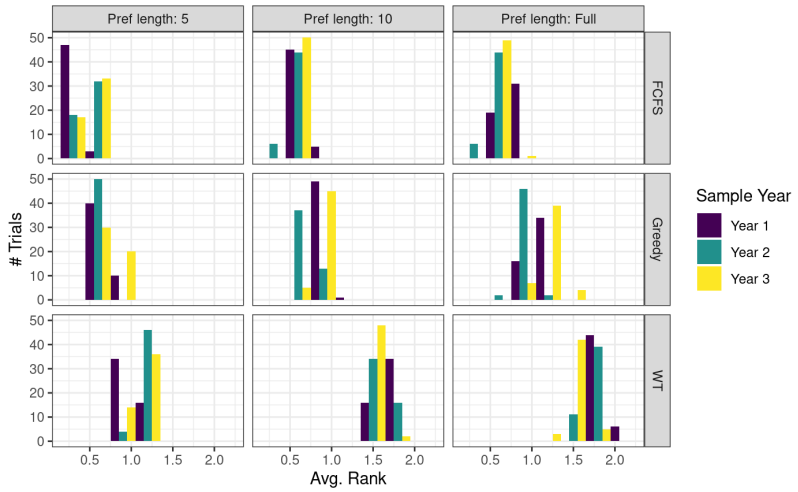
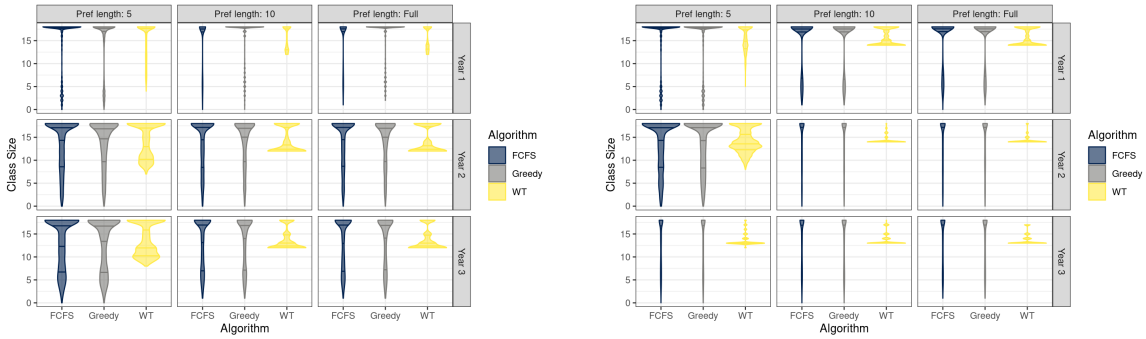


Figure 4.6: Comparison of average rank of assignment in each trial.

When we compare Figures 4.5 and 4.6, we find that the slightly higher number of unmatched students in GREEDY noticeably affects the average rank for students across trials, which indicates that students are the less satisfied in GREEDY than FCFS. Additionally, when we look at Figure 4.4 as well, we can see that there is a consequence in WT for having so many more blocking pairs, in that students on average receive a section they rank slightly lower than their assignments in both other algorithms. However, WT does still tend to give students their second or third choice across all years and preference list lengths, which we deem a worthy tradeoff for being the only algorithm that always makes a full matching.

4.3.3 Is it balanced?



(a) (b)

Figure 4.7: (a) Comparison between sample years of distribution of *grouped* students to class sections over all trials. (b) Comparison between sample years of distribution of all students to class sections over all trials *without group fairness constraints*.

Finally, though it was not explicitly part of the design of the algorithms, we are interested in how well the sections are balanced overall. Generally, we would hope all the sections are similarly sized. Figure 4.7a illustrates the distribution of students in each sample year when we run the algorithm as-is with student groups. We also isolated how well the algorithms balance class size by performing the trials with all students in the same group (Figure 4.7b).

The most successfully balanced trials are those with longer preference lists, and WT was the best at balancing by far across all trials. In most of the WT grouped trials, the smallest class is approximately size 12 and the largest class is size 18, giving a range of 6 for all three years of data. Especially in years 2 and 3, the majority of classes were size 12 or 13. Compared to FCFS and GREEDY, which both output class sizes ranging from 0 to 18, the WT output is definitively effective at balancing. If we compare Figure 4.7b to Figure 4.7a, we can see that class size becomes even more balanced when we do not consider groups for WT. As is guaranteed by Lemmas 3.1 and 3.2, FCFS does not exhibit a difference between the two figures, and GREEDY behaves the same as FCFS in the trials without student groups.

As a whole, the WT algorithm outperformed FCFS and GREEDY on every metric we analyzed other than blocking pairs and rank, especially in trials with longer preference lists. Due to our re-framing of the problem under intra-group stability from Definition 2.13, we accept this result as successful as we prioritize finding a full, group fair matching.

Conclusion

Our results show that the WT algorithm can be an effective potential solution for The Registrar to ensure all freshmen are registered for a year-long HUM 110 section. Our algorithm also contributes to the study of stable bipartite one-sided matching with incomplete preferences and group fairness constraints. By formulating the task as a weighted graph matching problem, we are able to give theoretical guarantees about every student being matched to a HUM section in a full, fair, intra-group stable solution. Though students may be assigned to a class section that is ranked lower on their preference list than they would get in the current system, our experiments show that students still tend to receive one of their top three preferences for the tradeoffs of group fairness and ensuring every student is matched.

In our experiments, we notably do not evaluate the WT algorithm against adversarial instances of FCFS and GREEDY. For both of those algorithms where the outcome is heavily dictated by the registration order, there necessarily exists certain orders of students that would result in a worst-case matching. Finding those theoretical worst-case orderings and testing performance against those cases would be a relevant extension to this project.

Additionally, our experiments were performed on partially synthesized data based on previous years of class scheduling, where the system may have inherent biases (such as those mentioned in Section 4.2.1) that would affect the performance of the algorithm in an actual implementation setting. It would also usually be standard practice to test the ϕ value in the Mallows preference synthesis to evaluate how the algorithm performs on different levels of deviation from the central vote. We determined that due to how many other variables we were already testing (data sample year, preference list length, randomness over $\phi = 0.4$, various algorithms), adding another was beyond the scope of this thesis. That being said, due to all these synthesis factors, testing WT in a real setting with sampled preferences would be helpful for confirming whether it is ready to be implemented as an actual mechanism at Reed.

Further expansions on this work could take many directions due the various different fields of computer science and mathematics this project incorporates. The WT algorithm produces many inter-group blocking pairs in each output; one could investigate approximation algorithms or heuristic methods of reducing the number of blocking pairs to achieve inter-group stability more often and compare the effective-

ness of those solutions to WT. We also do not explicitly design our algorithm around balancing class sizes beyond the enforcement of class capacities. There could be much more done with the weight function to attempt to encode the class size balancing as an additional constraint to the problem. Furthermore, most mechanism design solutions across stable matching and fair division are evaluated under *strategy-proofness*, or whether an entity in the model can "game" the system to achieve a better outcome for themself at the cost of other entities. We do not evaluate the WT algorithm for strategy-proofness; extending this project to find a strategy-proof solution would also be a fruitful direction for further study.

In conclusion, the WT algorithm we've developed in this thesis could viably be implemented as an actual mechanism at Reed to help The Registrar's Office solve their HUM-assignment problem. Additionally, we hope our project sets a precedent for future CS seniors to collaborate with The Registrar or other Reed institutions on projects that allow students to use their Reed education to make a lasting, tangible impact on the Reed community. We also wish all current and future Reed students luck in creating their class schedules for years to come.

References

[1] Michael Sipser. *Introduction to the Theory of Computation*. 2nd ed. Course Technology, 2006.

[2] Luc Longpré and Pierre McKenzie. "The Complexity of Solitaire". In: *Mathematical Foundations of Computer Science 2007*. Ed. by Luděk Kučera and Antonín Kučera. Berlin, Heidelberg: Springer Berlin Heidelberg, 2007, pp. 182–193.

[3] H. W. Kuhn. "The Hungarian method for the assignment problem". In: *Naval Research Logistics Quarterly* 2.1-2 (1955), pp. 83–97. DOI: https://doi.org/10.1002/nav.3800020109.

[4] William S. Zwicker. "Introduction to the Theory of Voting". In: *Handbook of Computational Social Choice*. Ed. by Felix Brandt et al. Cambridge University Press, 2016, pp. 23–56.

[5] William Vickrey. "COUNTERSPECULATION, AUCTIONS, AND COMPETITIVE SEALED TENDERS". In: *The Journal of Finance* 16.1 (1961), pp. 8–37. DOI: https://doi.org/10.1111/j.1540-6261.1961.tb02789.x.

[6] Robert W. Irving. "An efficient alorithm for the "stable roommates" problem". In: *Journal of Algorithms* 6.4 (1985), pp. 577–595. DOI: https://doi.org/10.1016/0196-6774(85)90033-1.

[7] Martin Aleksandrov et al. "Online Fair Division: analysing a Food Bank problem". In: (2015). arXiv: 1502.07571 [cs.GT]. URL: https://arxiv.org/abs/1502.07571.

[8] Alvin E. Roth. "The Origins, History, and Design of the Resident Match". In: *JAMA* 289.7 (Feb. 2003), pp. 909–912. DOI: https://doi.org/10.1001/jama.289.7.909.

[9] Jon Kleinberg and Éva Tardos. *Algorithm Design: 1. Stable Matching*. Lecture Notes. 2005. URL: https://www.cs.princeton.edu/~wayne/kleinberg-tardos/pdf/01StableMatching.pdf.

[10] David Gale and Lloyd S. Shapley. "College admissions and the stability of marriage." In: *American Mathematical Monthly* 69 (1962), pp. 9–15. DOI: 10.4236/tel.2012.23054.

[11] *The Holy Bible: King James Version*. Genesis 13:9. 1611. Chap. 13.

[12] Sean Ingham. *Pareto-optimality*. URL: https://www.britannica.com/money/Pareto-optimality.

[13] Péter Biró et al. "The College Admissions problem with lower and common quotas". In: *Theoretical Computer Science* 411.34 (2010), pp. 3136–3153. DOI: https://doi.org/10.1016/j.tcs.2010.05.005.

[14] Santhini K. A., Raghu Ravi, and Meghana Nasre. "Matchings under one-sided preferences with soft quotas". In: (Mar. 2025). DOI: 10.2139/ssrn.5163631.

[15] Isa E. Hafalir, M. Bumin Yenmez, and Muhammed A. Yildirim. "Effective affirmative action in school choice". In: *Theoretical Economics* 8.2 (2013), pp. 325–363. DOI: https://doi.org/10.3982/TE1135.

[16] Thành Nguyen and Rakesh Vohra. "Stable Matching with Proportionality Constraints". In: *Operations Research* 67.6 (2019), pp. 1503–1519. DOI: 10.1287/opre.2019.1909.

[17] Cynthia Dwork et al. "Fairness Through Awareness". In: *CoRR* abs/1104.3913 (2011). arXiv: 1104.3913. URL: http://arxiv.org/abs/1104.3913.

[18] Atila Abdulkadiroğlu and Tayfun Sönmez. "School Choice: A Mechanism Design Approach". In: *American Economic Review* 93.3 (June 2003), pp. 729–747. DOI: 10.1257/000282803322157061.

[19] Atila Abdulkadiroğlu, Parag A. Pathak, and Alvin E. Roth. "Strategy-Proofness versus Efficiency in Matching with Indifferences: Redesigning the NYC High School Match". In: *American Economic Review* 99.5 (Dec. 2009), pp. 1954–78. DOI: 10.1257/aer.99.5.1954.

[20] Ryoji Kurata et al. "Controlled school choice with soft bounds and overlapping types". In: *Journal of Artificial Intelligence Research* 58 (Jan. 2017), pp. 153–184. DOI: 10.1613/jair.5297.

[21] Eric Budish and Estelle Cantillon. "The Multi-unit Assignment Problem: Theory and Evidence from Course Allocation at Harvard". In: *American Economic Review* 102.5 (May 2012), pp. 2237–71. DOI: 10.1257/aer.102.5.2237.

[22] Santhini K. A. et al. "Group Fairness and Multi-Criteria Optimization in School Assignment". In: *6th Symposium on Foundations of Responsible Computing (FORC 2025)*. Ed. by Mark Bun. Vol. 329. Leibniz International Proceedings in Informatics (LIPIcs). Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2025, 20:1–20:20. DOI: 10.4230/LIPIcs.FORC.2025.20.

[23] Atasi Panda et al. "Group Fair Matchings using Convex Cost Functions". In: (2025). arXiv: 2508.12549 [cs.GT]. URL: https://arxiv.org/abs/2508.12549.

[24] Atila Abdulkadiroglu. "Generalized matching for school choice". In: *Unpublished paper, Duke University. [1311, 1312]* (2011).

[25] Reed College. *Group Requirements - Office of the Registrar*. URL: https://www.reed.edu/registrar/academic-advising-handbook/sections/group-requirements.html.

[26] C. L. Mallows. "Non-Null Ranking Models. I". In: *Biometrika* 44.1/2 (1957), pp. 114–130. DOI: https://doi.org/10.1093/biomet/44.1-2.114.

[27] Niclas Boehmer et al. *Guide to Numerical Experiments on Elections in Computational Social Choice*. 2024. arXiv: 2402.11765 [cs.GT]. URL: https://arxiv.org/abs/2402.11765.

[28] Niclas Boehmer and Nathan Schaar. *Collecting, Classifying, Analyzing, and Using Real-World Elections*. 2023. arXiv: 2204.03589 [cs.GT]. URL: https://arxiv.org/abs/2204.03589.

[29] Pauli Virtanen et al. "SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python". In: *Nature Methods* 17 (2020), pp. 261–272. DOI: 10.1038/s41592-019-0686-2.

[30] Roy Jonker and Ton Volgenant. "A shortest augmenting path algorithm for dense and sparse linear assignment problems". In: *Computing* 38 (1987), pp. 325–340. DOI: https://doi.org/10.1007/bf02278710.